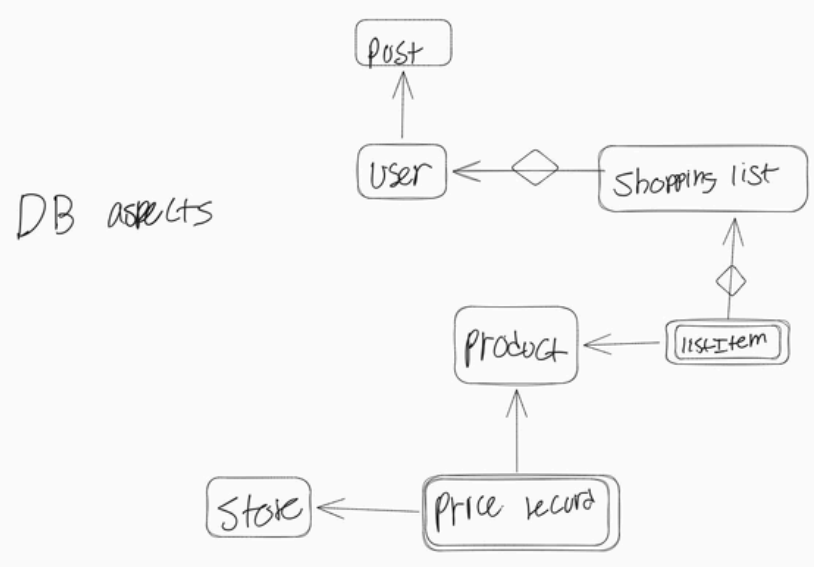

ER MODEL Table/Conversion/Normization

GroceryApp (
user_id, username, email, pw, fname, lname,
list_id, list_name, is_active, create_at,
item_id, quantity, checked,
product_id, product_name, category, brand, unit_size, unit_type,
store_id, store_name, chain, city, zip,
record_id, price, reg_price, is_sale, price_date,
post_id, title, body, posted_at
)

FDS

user_id → user_name,email,pw,fname,lname
user_name -> user_id
email → user_id, username, password_hash, firstName, lastName
store_id → store_name, chain, city, zip
record_id → product_id,store_id,price,reg_price,is_sale,price_date
list_id → user_id,list_name,is_active,create_at
post_id → user_id,title,body,posted_at
item_id → list_id,product_id,quanitity,checked
product_id → product_name,catergory, brand,unit_size,unit_type
(product_id, store_id, price_date) → price, reg_price, is_sale

Comparison Between ER Model and Normalization

The Two approaches have produce very
similar/identical tables

user_id → username, email, pw, fname lname
R1(user_id, username, email, pw, fname, lname)
user_id → all attributes
username → user_id
R_1 is in BCNF

R_2(user_id,list_id, list_name, is_active, create_at,item_id, quantity, checked,
product_id, product_name, category, brand, unit_size, unit_type,
store_id, store_name, chain, city, zip,record_id, price, reg_price, is_sale,
price_date,post_id,title,body,posted_at)

store_id → store_name, chain, city, zip
R3(store_id, store_name, chain, city, zip)

R_2(user_id,list_id, list_name, is_active, create_at,item_id, quantity, checked,
product_id, product_name, category, brand, unit_size, unit_type,store_id,record_id, price, reg_price,
is_sale, price_date,post_id, title, body, posted_at)

product_id → product_name, category, brand, unit_size, unit_type
R4 (product_id, product_name, category, brand, unit_size, unit_type)

R_2(user_id,list_id, list_name, is_active, create_at,item_id, quantity, checked,product_id,
store_id,record_id, price, reg_price, is_sale, price_date,post_id, title, body, posted_at)

list_id → user_id, list_name, is_active, create_at
R5(list_id, user_id, list_name, is_active, create_at)

R_2(list_id,item_id, quantity, checked,product_id,store_id,
record_id, price, reg_price, is_sale, price_date,post_id, title, body, posted_at)

item_id → list_id, product_id, quantity, checked
R6(item_id, list_id, product_id, quantity, checked)

R_2(item_id,product_id,store_id,record_id, price, reg_price, is_sale, price_date,post_id, title,
body,posted_at)

post_id → user_id, title, body, posted_at
R7(post_id, user_id, title, body, posted_at)

R_2(item_id,product_id,store_id,record_id, price, reg_price, is_sale, price_date,post_id)

record_id → product_id, store_id, price, reg_price, is_sale, price_date
R8(record_id, product_id, store_id, price, reg_price, is_sale, price_date)

R_2(item_id, product_id, store_id, record_id, post_id)
item_id → product_id
record_id → product_id, store_id

candidate key ~ (item_id, record_id, post_id)

Result:
After applying BCNF decomp to the wide table, the remainder R2 contained only foreign key column objs ~ (item_id, product_id, store_id, record_id, post_id) with no true meaningful or descriptive attribute to tie them into a realworld relationship. the remainder is an artifact of combining unrelated entities into a single wide table, and doesn't accurately represent a grocery domain, but was able to be reduced further to identify a candidate key. Regardless, the remaining R1-R8 were captured for our DB and closely resembled our pre-coordinated ER model, which we had designed with thorough consideration.